

时钟系统

结构示意图:

时钟源

芯片内部集成有
8M RC振荡器为系统提供时钟,
开发板外接8M晶振,可产生更稳定的时钟信号

提速

对时钟源信号经分/倍频
处理所得的时钟信号

系统时钟
最高 12M

耗电、电磁干扰大

分级降速

提供时钟信号

根据外设的需求分频
为不同速度的时钟
信号

各级总线及
外设

STM32有5个时钟源:

① HSI: 高速内部时钟, RC振荡器, 频率 8MHz, 精度不高

② HSE: 高速外部时钟, 可接石英/陶瓷谐振器, 或接外部时钟源, 频率 4MHz ~ 16MHz

③ LSI: 低速内部时钟, RC振荡器, 频率 40kHz, 提供低功耗时钟
→ 也是独立看门狗(WDG)的时钟源

④ LSE: 低速外部时钟, 接频率 32.768 kHz 的石英晶体
= 2^{15} , 经分频可产生 1Hz 的信号
需要精确秒信号
→ 可为 RTC (实时时钟) 产生时钟信号

⑤ PLL: 锁相环倍频输出, 时钟输入源可选为 HSI/2、HSE 或 HSE/2,

倍频可选 2~16 倍, 但其输出频率最大不得超过 72MHz
自身并不能产生时钟信号, 要在 HSI、HSE 的基础上倍频才可产生

• 系统时钟 SYSCLK 可来源于 3 个时钟源:

- ① HSI 振荡器时钟
- ② HSE 振荡器时钟
- ③ PLL 时钟

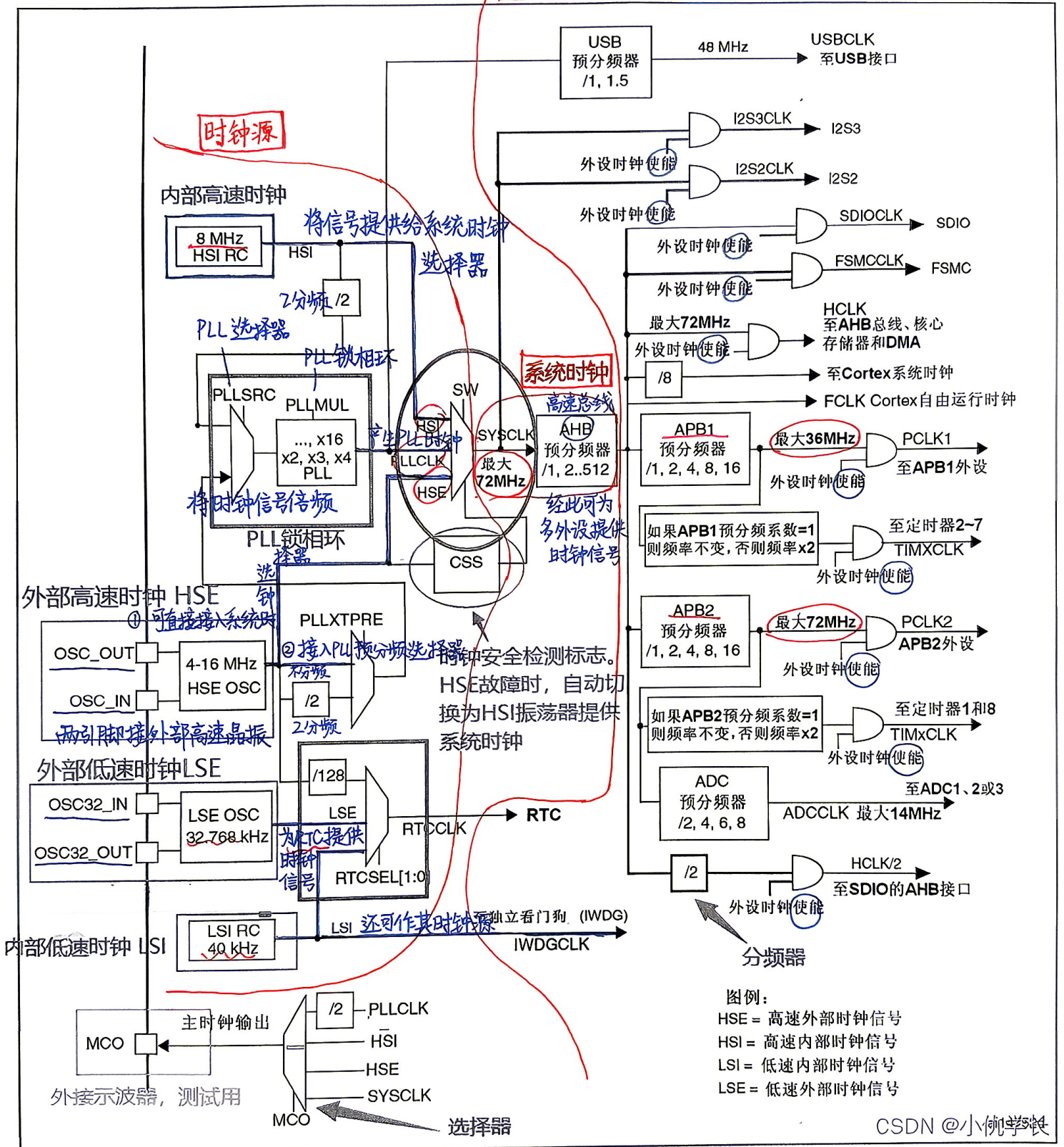
• PA8 引脚可作为时钟输出引脚, 为外部设备提供时钟信号或同步信号

STM32 可选择 1 个时钟信号输出到 MCO 脚 (PA8) 上, 可选择为 PLL 输出的 2 分频、HSI、HSE、或系统时钟

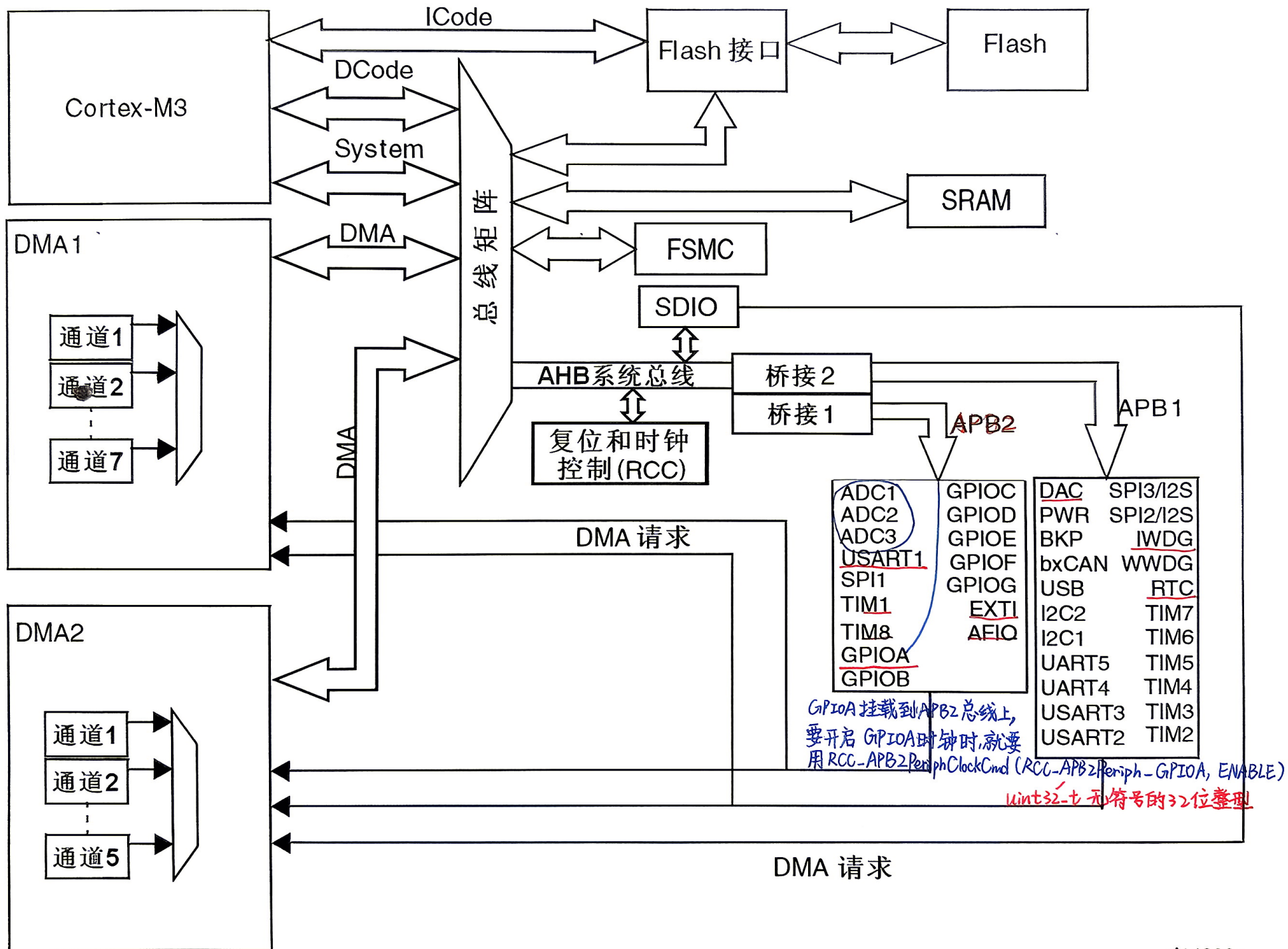
• 时钟系统是系统的脉搏, 任何外设使用前, 务必先使能其相应的时钟

图8 时钟树

各级总线及外设 所有外设都要先使能才可工作



内部RC振荡器 → 时钟源
外部晶振 →



GPIO 工作原理

无协议的通信

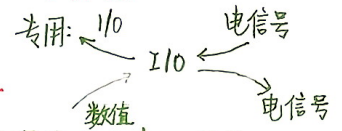
I/O只呈现高低电平(1/0)这2种状态

作用通用目的的输入或输出功能

芯片上所有外设的配置和功能都通过寄存器来实现:

本质: 对寄存器的每个位进行位操作

库函数就是封装好了寄存器操作的函数



当用作输出功能时, 通过输出高/低电平来表示程序中的逻辑1/0

当用作输入功能时, 程序通过查询对应寄存器的值是1/0来查询外部输入引脚是高/低电平

注意: 高/低电平是数字逻辑的概念, 它们表示的是电压的范围, 而非具体某个值

当引脚电压高于2V时, 是高电平; 当引脚电压低于0.8V时, 是低电平

STM32F103ZET6

一共有7组I/O口 (GPIOA ~ GPIOG)

每个I/O口有0~15共16个引脚

每组I/O口有16个I/O, 共 $7 \times 16 = 112$ 个I/O

GPIO 工作模式

输入模式 { 输入浮空, 输入上拉, 输入下拉, 模拟输入 }

得数字量0,1

输出模式 { 开漏输出, 推挽输出, 复用开漏输出, 复用推挽输出 }

翻转速度

2MHz, 10MHz, 50MHz

得模拟量, 中间值有意义, 不能用0,1

ADC: 将模拟量转换为数字量

若非输出数据寄存器而是片上外设控制输出, 就是“复用”

输出速度的含义: 输出引脚每一秒中高/低电平翻转的次数

输入浮空

将外部引脚的高/低电平反映到输入数据寄存器对应的位上(1/0)

不上拉, 不下拉

输入下拉

对引脚 浮空状态

当外部元器件没有输入时, IDR中的值读到的是0

当外部对引脚输入高电平时, IDR中的值读到的是1



输入上拉

浮空状态

当外部对GPIO引脚没有输入时, IDR中的值读到的是1

当外部对GPIO引脚输入低电平时, IDR中的值读到的是0



模拟输入

GPIO 不具备模拟信号处理能力, 故外部器件传送给引脚的模拟信号一般是传送给 ADC 等具备模拟信号处理能力的功能模块

由于模拟信号是连续变化的信号量, 所以在数据通路中不能接上/下拉电阻, 也不能接有滤波能力的 TTL 施密特触发器.

开漏输出 $\Rightarrow$ 只可以输出低电平 要输出高电平, 就要外接上拉电阻

只接入 N-MOS 管而不接入 P-MOS 管

当 ODR 中的值是 1 时, 输出控制电路将转换为高电平, 由于只接了 N-MOS 管, 故对高电平是截止的. 此时外部引脚没有任何动作, 外部电平状态由引脚的上/下拉电阻决定.

输出模式中, 输入的数据模式也是接通的, 故 IDR 中可以读到输出状态值
通路 4 种输出都适用

当 ODR 中的值是 0 时, 输出控制电路将转换为低电平, 此时 N-MOS 管导通, GPIO 引脚的状态下拉到 V_{SS} (低电平), 此时外部输出低电平, IDR 中也可以读到输出的电平状态: 低电平

复用开漏输出电路原理与之类似

不同之处在于:

GPIO 开漏输出时, 输出状态由 ODR 决定

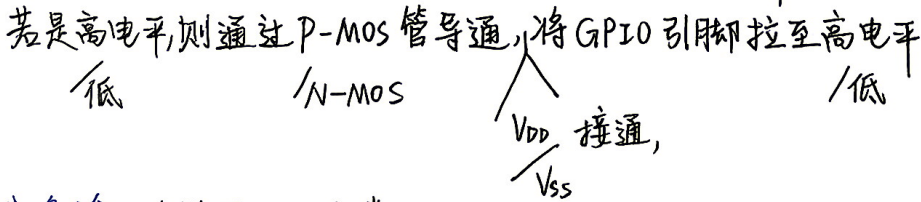
复用开漏输出时, 输出状态由复用功能决定

推挽输出 $\Rightarrow$ 可以输出高/低电平

可以输出强高/低电平, 连接数字器件

同时接入 N-MOS 管和 P-MOS 管

MCU 通过位设置/清除寄存器或直接读写的方式, 将待输出的数据写入 ODR, 输出控制电路会将 ODR 中的值转换为高/低电平



复用推挽输出电路原理与之类似

不同之处在于:

GPIO 推挽输出时, 输出状态由 ODR 决定

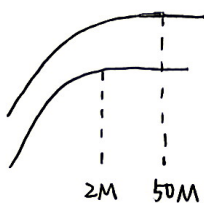
复用推挽输出时, 输出电平状态由片上外设决定

只可以输出强低电平, 高电平只能靠外部电阻拉高.

输出端相当于三极管的集电极, 要得到高电平状态需要上拉电阻.

适合做电流型驱动, 吸收电流能力相对较强 (20 ma 以内, 一般来说)

3 种输出速度: 2MHz、10MHz、50MHz



虽然有输入浮空模式,可输入高/低电平,可依旧需要上/下拉输入,理由:

某些传感器等元器件,信号输出有效时会输出高(低)电平,无效时不输出,为开路,此时引脚处于浮空状态,为了保证不输出信号时,也有一个明确的电平状态,所以要接一个与有效电平相反的电阻
低(高) 下(上)拉

GPIO 输入模式的选择:

- 外部设备既可输入高电平,又可输入低电平时,选择浮空模式
- 外部设备只可输入高电平时,选择下拉模式
- 外部设备只可输入低电平时,选择上拉模式

GPIO 寄存器说明

每组GPIO端口的寄存器包括:

对低8个引脚配置: 0~7 1)

每组16个引脚, $16 \times 4 = 64$ (位)

对高8个引脚配置: 8~15 2)

$\frac{64}{32} = 2$ (个)

① 2个32位配置寄存器 (GPIOx-CRL, GPIOx-CRH)

1个GPIO端口有16个引脚,配置每个引脚要4位寄存器,一个需要64位,

而MCU是32位的芯片,所以需要2个。

② 2个32位数据寄存器 (GPIOx-IDR, GPIOx-ODR)

表示外部引脚电平状态

决定对应引脚的输出电平状态

☆ 由于一个引脚只有高/低两种电平状态,因此用一个数据位即可表示,一个位对应一个引脚

③ 1个32位置位/复位寄存器 (GPIOx-BSRR)

对ODR中数据位的设置

对某一个数据位进行置位(写1)

对某一个数据位进行复位(清零)

④ 1个16位复位寄存器 (GPIOx-BRR)

对ODR的复位, BRR中一个位对应ODR中一个位

⑤ 1个32位锁定寄存器 (GPIOx-LCKR)

引脚功能的锁定

共7个寄存器,完成了GPIO的功能

- 每个I/O端口位可自由编程,每个位数值可自由配置,与其他位无关
- I/O端口寄存器必须按32位字被访问,即:必须对整个寄存器进行操作,不允许半字或字节访问。

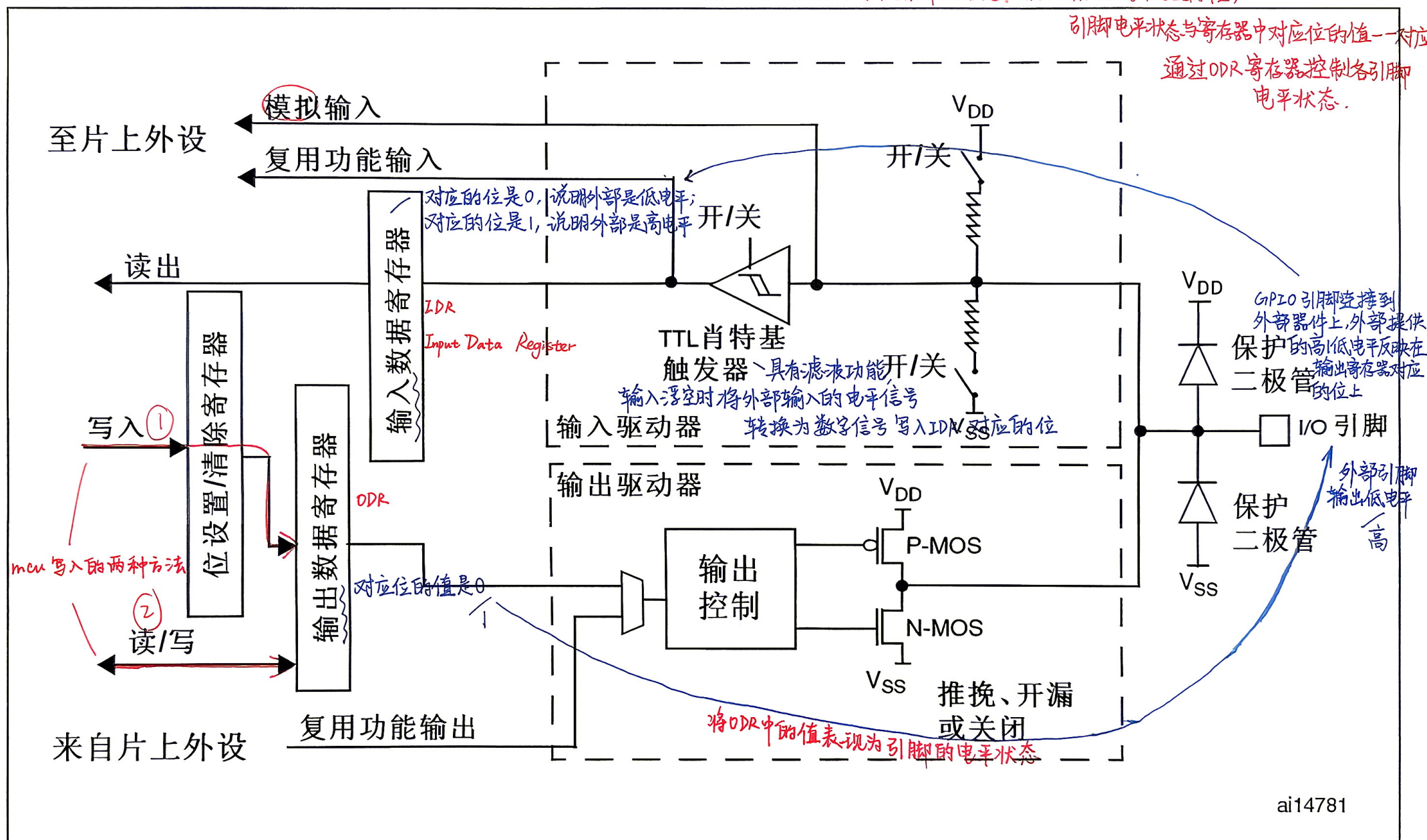
初始化结构体的成员变量就是描述外设的属性系数

1个机器字就是32位,4个字节

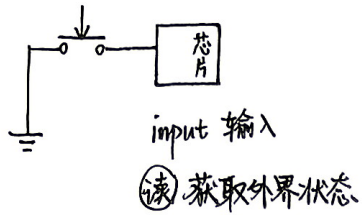
一组GPIO端口有16个引脚，输入输出时16个数据位即可表示每个引脚的状态，我们操纵寄存器的位，

引脚电平状态与寄存器中对应位的值一一对应

通过ODR寄存器控制各引脚电平状态。



没按键按下,输入0



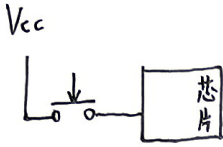
有低的,就要补1: 设置上拉(没按下自动呈现高电平)

目的: 为明确输入信号是高电平还是低电平,

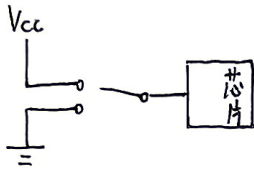
某些情况需补充另一种信号

要求: 保证信号完整的前提下, 还要不影响输入状态、

同理:



有1, 要补0, 设置下拉



设置浮空即可

1个寄存器位对应1个引脚的输入/输出状态

IDR/ODR: 16位寄存器, 16 bit

不是1个数字, 而是16个引脚输入状态的集合

不能对寄存器做数学运算, 可按位获取状态

① 六种位运算

运算符	含义
&	按位与
	按位或
^	按位异或
~	取反
<<	左移
>>	右移

1个位: 1 bit $\begin{matrix} 0 \\ \swarrow \\ 1 \end{matrix}$

1个变量里有16个位, 分配每1位去管理1个引脚

此时寄存器里存放的是16个引脚的状态的集合.

$+-*\div$ 无意义, 位运算有意义.

将第4~7位清0: $\text{GPIOA} \rightarrow \text{CRL} \&= 0\text{FFFFFFF0F};$

设置相应位的值, 不改变其他位的值: $\text{GPIOA} \rightarrow \text{CRL} |= 0\text{X00000040};$

将1左移5位(即将位5置1): $\text{GPIOA} \rightarrow \text{ODR} |= 1<<5;$

将TIMx的状态寄存器SR中的TIM-FLAG对应的标志位清零: $\text{TIMx} \rightarrow \text{SR} = (\text{uint16_t}) \sim \text{TIM-FLAG};$

② #define 标识符 字符串 (预处理命令, 用于宏定义)

"标识符"为所定义的宏名, "字符串"可以是常数、表达式、格式串等.

③ extern 变量声明

可置于变量或函数前, 以表示变量或函数的定义在其他文件中, 提示编译器遇该变量和函数时在其他模块中寻找定义.

注意: extern 声明变量可以多次, 但定义仅一次.

Main.c 文件

```
u8 id; // 定义(仅允许1次)
```

```
main ( )
```

```
{
```

```
    id = 1;
```

```
    printf("%d", id); // id=1
```

```
    test();
```

```
    printf("%d", id); // id=2
```

```
}
```

test.c 文件

```
extern u8 id;
```

```
void test(void)
```

```
{
```

```
    id = 2;
```

```
}
```

④ typedef 类型别名

定义一种类型的别名,而非简单的宏替换,可以用作同时声明指针型的多个对象.

```
typedef unsigned char    uint8_t;
typedef unsigned short int uint16_t;
typedef unsigned int      uint32_t;
typedef unsigned int 64   uint64_t;
```

⑤ 结构体: 构造类型

```
struct 结构体名
{
    成员列表1;
    成员变量2;
    ...
} 变量名列表;
```

结构体申明时可以定义变量,也可申明之后定义,方法是:

struct 结构体名 结构体变量列表;

同一类型可以用数组,不同类型可以用结构体去组织

例如: void GPIO_Init (GPIO_TypeDef * GPIOx, GPIO_InitTypeDef * GPIO_InitStruct)

⑥ static 关键字

static 声明的局部变量,存储在静态存储区,在函数调用结束后不会被释放,它的值会一直保留下来,具有记忆功能.

```
int GetValue(void)
{
    int flag = 0;
    flag++;
    return flag;
}
```

```
int GetValue(void)
{
    static int flag = 0;
    flag++;
    return flag;
}
```

第1次调用结果: |

2 |

3 |

⋮ |

n |

|

2

3

⋮

n

→ 对某个GPIO端口的低8个引脚进行配置(0~7)

8.2.1 端口配置低寄存器(GPIOx_CRL) (x=A..E)

相对于基地址而言 偏移地址: 0x00

32位, 4字节

复位值: 0x4444 4444 → 复位后, 每个引脚对应的4个位的寄存器中写入的值是4, 即0100

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

以0为例, 这是对0号引脚进行配置
2个MODE位, 2个CNF位

位31:30	CNFy[1:0]: 端口x配置位(y = 0...7) (Port x configuration bits) 软件通过这些位配置相应的I/O端口, 请参考表17端口位配置表。 在输入模式(MODE[1:0]=00): 00: 模拟输入模式 01: 浮空输入模式(复位后的状态) 10: 上拉/下拉输入模式 11: 保留 在输出模式(MODE[1:0]>00): 00: 通用推挽输出模式 01: 通用开漏输出模式 10: 复用功能推挽输出模式 11: 复用功能开漏输出模式
27:26	
23:22	
19:18	
15:14	
11:10	
7:6	
3:2	
位29:28	MODEy[1:0]: 端口x的模式位(y = 0...7) (Port x mode bits) 软件通过这些位配置相应的I/O端口, 请参考表17端口位配置表。 00: 输入模式(复位后的状态) 01: 输出模式, 最大速度10MHz 10: 输出模式, 最大速度2MHz 11: 输出模式, 最大速度50MHz
25:24	
21:20	
17:16	
13:12	
9:8, 5:4	
1:0	

CNF的2个位

如何区分?

当 CNF1 CNF0 MODE1 MODE0 时
1 0 0 0

若ODR中对应的寄存器的位的值是1, 就是上拉;
若对应位的值是0, 就是下拉。

MODE的2个位

输入模式时, MODE位
配置为00

→ 对某个GPIO端口的高8个引脚进行配置(8~15)

8.2.2 端口配置高寄存器(GPIOx_CRH) (x=A..E)

偏移地址: 0x04

复位值: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:30	CNFy[1:0]: 端口x配置位(y = 8...15) (Port x configuration bits) 软件通过这些位配置相应的I/O端口, 请参考表17端口位配置表。 在输入模式(MODE[1:0]=00): 00: 模拟输入模式 01: 浮空输入模式(复位后的状态) 10: 上拉/下拉输入模式 11: 保留 在输出模式(MODE[1:0]>00): 00: 通用推挽输出模式 01: 通用开漏输出模式 10: 复用功能推挽输出模式 11: 复用功能开漏输出模式
27:26	
23:22	
19:18	
15:14	
11:10	
7:6	
3:2	
位9:28	MODEy[1:0]: 端口x的模式位(y = 8...15) (Port x mode bits) 软件通过这些位配置相应的I/O端口, 请参考表17端口位配置表。 00: 输入模式(复位后的状态) 01: 输出模式, 最大速度10MHz 10: 输出模式, 最大速度2MHz 11: 输出模式, 最大速度50MHz
25:24	
21:20	
17:16	
13:12	
9:8, 5:4	
1:0	

8.2.3 端口输入数据寄存器(GPIOx_IDR) (x=A..E)

地址偏移: 0x08

复位值: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位31:16	保留, 始终读为0。
位15:0	IDRy[15:0]: 端口输入数据(y = 0...15) (Port input data) 这些位为只读并只能以字(16位)的形式读出。读出的值为对应I/O口的状态。

每个GPIO端口有16个引脚, 每个引脚只有2种状态(高, 低电平),
故需16个位来表示所有引脚状态。

当引脚为输出状态时, 控制引脚的输出电平

8.2.4 端口输出数据寄存器(GPIOx_ODR) (x=A..E)

地址偏移: 0Ch

复位值: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:16	保留, 始终读为0。
位15:0	ODRy[15:0]: 端口输出数据(y = 0...15) (Port output data) 这些位可读可写并只能以字(16位)的形式操作。 注: 对GPIOx_BSRR(x = A...E), 可以分别地对各个ODR位进行独立的设置/清除。

每个GPIO 端口有16个引脚, 每个引脚只有:
故需16个位来表示所有引脚的输出电平

8.2.5 端口位设置/清除寄存器(GPIOx_BSRR) (x=A..E)

地址偏移: 0x10

复位值: 0x0000 0000

Bit 位
Set Reset Register

为0时, 都不起作用, 不对 ODR 对应位产生影响
当 BS1 为1时, 对 PA1 引脚执行 Set 功能, 对应的 ODR 位是1, 使 PA1 输出高电平
低

这16个寄存器位
执行 Reset 功能

这16个寄存器位
执行 Set 功能

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
W	W	W	W	W	W	W	W	W	W	W	W	W	W	W	W
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
W	W	W	W	W	W	W	W	W	W	W	W	W	W	W	W

位31:16	BRy : 清除端口x的位y (y = 0...15) (Port x Reset bit y) 这些位只能写入并只能以字(16位)的形式操作。 0: 对对应的ODRy位不产生影响 1: 清除对应的ODRy位为0 注: 如果同时设置了BSy和BRy的对应位, BSy位起作用。
位15:0	BSy : 设置端口x的位y (y = 0...15) (Port x Set bit y) 这些位只能写入并只能以字(16位)的形式操作。 0: 对对应的ODRy位不产生影响 1: 设置对应的ODRy位为1

例如, 要将 ODR 的位0 ReSet, 就要在 BR0 写入1 使之起效, 结果是0
Set BS0 1

法一: GPIOA → BSRR = 1 1步操作即可完成

GPIOA → ODR = 1 读数据至CPU的寄存器

法二: GPIOA → ODR = 0x1 3步 改写回去

8.2.6 端口位清除寄存器(GPIOx_BRR) (x=A..E)

地址偏移: 0x14

复位值: 0x0000 0000

16位

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
W	W	W	W	W	W	W	W	W	W	W	W	W	W	W	W

高16位保留状态

低16位起作用

当某一位为1时,将会清除对应位的ODR寄存器.

为0时不起作用,无影响.

115/754



参照2009年12月 RM0008 Reference Manual 英文第10版

本译文仅供参考,如有翻译错误,请以英文原稿为准。请读者随时注意在ST网站下载更新版本

通用和复用功能I/O

STM32F10xxx参考手册

位31:16	保留。
位15:0	BRy : 清除端口x的位y (y = 0...15) (Port x Reset bit y) 这些位只能写入并只能以字(16位)的形式操作。 0: 对对应的ODRy位不产生影响 1: 清除对应的ODRy位为0

8.2.7 端口配置锁定寄存器(GPIOx_LCKR) (x=A..E)

当执行正确的写序列设置了位16(LCKK)时，该寄存器用来锁定端口位的配置。位[15:0]用于锁定GPIO端口的配置。在规定的写入操作期间，不能改变LCKP[15:0]。当对相应的端口位执行了LOCK序列后，在下次系统复位之前将不能再更改端口位的配置。

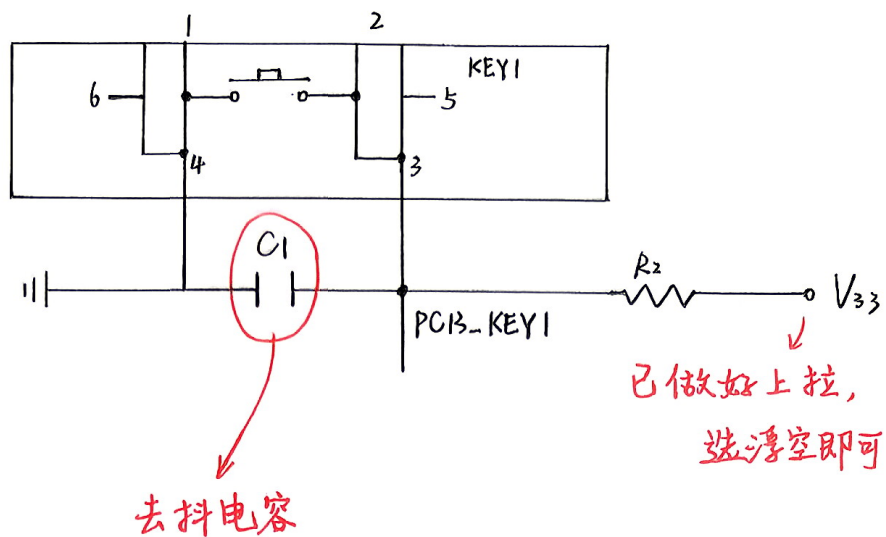
每个锁定位锁定控制寄存器(CRL, CRH)中相应的4个位。

地址偏移: 0x18

复位值: 0x0000 0000

[illegible]

位31:17	保留。
位16	LCKK: 锁键 (Lock key) 该位可随时读出，它只可通过锁键写入序列修改。 0: 端口配置锁键位激活 1: 端口配置锁键位被激活，下次系统复位前GPIOx_LCKR寄存器被锁住。 锁键的写入序列: 写1 -> 写0 -> 写1 -> 读0 -> 读1 最后一个读可省略，但可以用来确认锁键已被激活。 注：在操作锁键的写入序列时，不能改变LCK[15:0]的值。 操作锁键写入序列中的任何错误将不能激活锁键。
位15:0	LCKy: 端口x的锁位y (y = 0...15) (Port x Lock bit y) 这些位可读可写但只能在LCKK位为0时写入。 0: 不锁定端口的配置 1: 锁定端口的配置

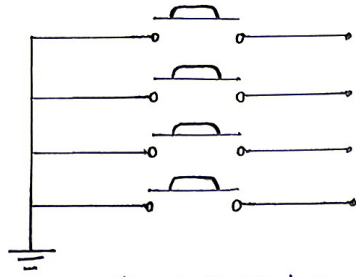


按键抖动时, 并联的这个电容接通了, 短时间内放电以保持电压不变

拓展: 电感 → 保持电流不变

GPIO 输入外设

独立按键



接GND, 对应的引脚配置输入上拉

(若接Vcc, 就要配置输入下拉)

(当外界给明确输入信号, 就设置浮空)

读取状态流程:

按下去是低电平

- 1) 使能按键对应IO口时钟, 调用函数: `RCC-APB2PeriphClockCmd()`;
- 2) 初始化IO模式: 上/下拉输入, 调用函数: `GPIO-Init()`;
- 3) 扫描IO口电平 (库函数/寄存器/位操作)

读取状态方法:

- 1) 读取IO口输入电平调用库函数为: `uint8-t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16-t GPIO_Pin);`
- 2) 读取IO口输入电平操作寄存器为: `GPIOx → IDR`: 端口输入寄存器
- 3) 使用位带操作读取IO口输入电平: `PEin(n)` - 读取 `GPIOE.n` 口电平

扫描按键方法:

- { 电平检测
- { 边沿检测

是否支持连按

```
u8 Key-Scan-C(void)
{
    if(PBin(0) == 0)
    {
        Delay-ms(15);
        if(PBin(0) == 0)
        {
            return 0;
        }
    }
    return 1;
}
```

延时 去抖

返回0表示按键按下
1 没

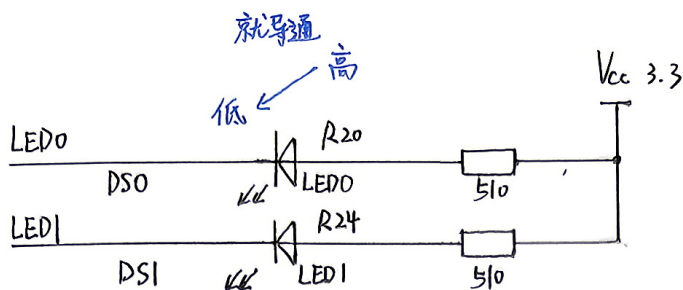
```
u8 Key-Scan(void)
{
    static u8 key-up = 1;
    if (key-up && (PBin(0) == 0))
    {
        Delay-ms(15);
        key-up = 0;
        if (PBin(0) == 0)
        {
            return 0;
        }
    }
    else if (PBin(0) == 1)
    {
        key-up = 1;
        return 1;
    }
}
```

也可放入

调用的频率就是返回的频率, 故支持连按.

GPIO 输出外设

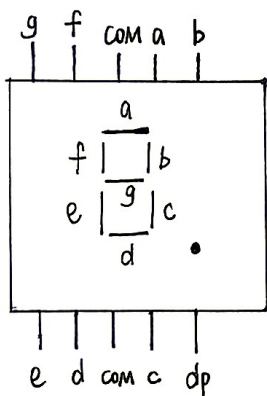
LED



用GPIO 控制，LED 是推挽输出

高电平熄灭
低电平点亮

数码管 → 其控制引脚可连到 mcu 的任意 GPIO 引脚上去



符号和引脚

设共阳极接法，点亮形成数字“1”

步骤：

① 连接到 GPIO 引脚

② mcu 控制引脚输出高低电平 (8 条语句)

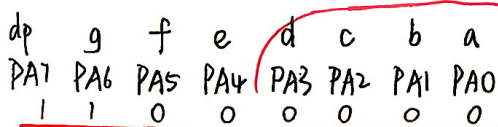
为每个引脚的输出状态写一个控制程序。

API 控制某一个端口的某一个引脚输出高/低电平 $API \rightarrow PortX.n$

更佳：同一端口的引脚的输出状态由同一 ODR 控制，最终作用到 ODR

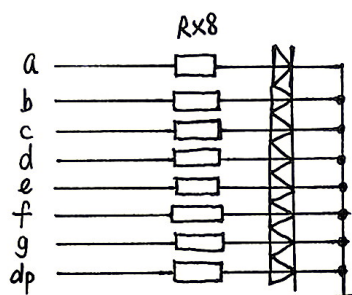
故将 a ~ dp 连到一组端口连续的引脚 PA0 ~ PA7 上

例如

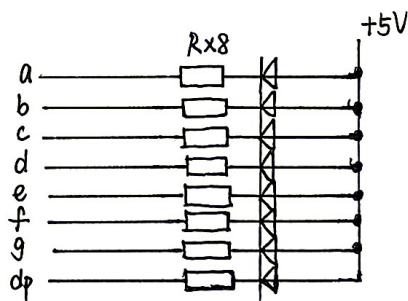


引脚输出电平状态的一个集合

低 8 位: 0XC0



高电平驱动
共阳极



低电平驱动
共阳极

0XC0	0X80
0XF9	0X90
0XA4	
0XB0	
0X99	
0X92	
0X82	
0XF8	

$GPIOA \rightarrow ODR \&= 00FF00$ // 清零

$GPIOA \rightarrow ODR |= 0X00C0$

编码方式很简洁!

多位数码管如何显示：

段选线 → 控制选择什么数字

PC0、PC1

位选线 → 控制选择哪一位点亮

PA0~PA7

按位刷新(显示)：每一位显示的时间要小于人眼分辨率(例如5ms)

执行1次

10ms

15 执行

100 次

① 两位位选

十位 个位
X X

② 将个位数字写入ODR

③ 开启个位的位选

④ 延时5ms

不然会有影响：若不把十位的关掉，本来该十位的显示就会跑到个位，出现“残影”

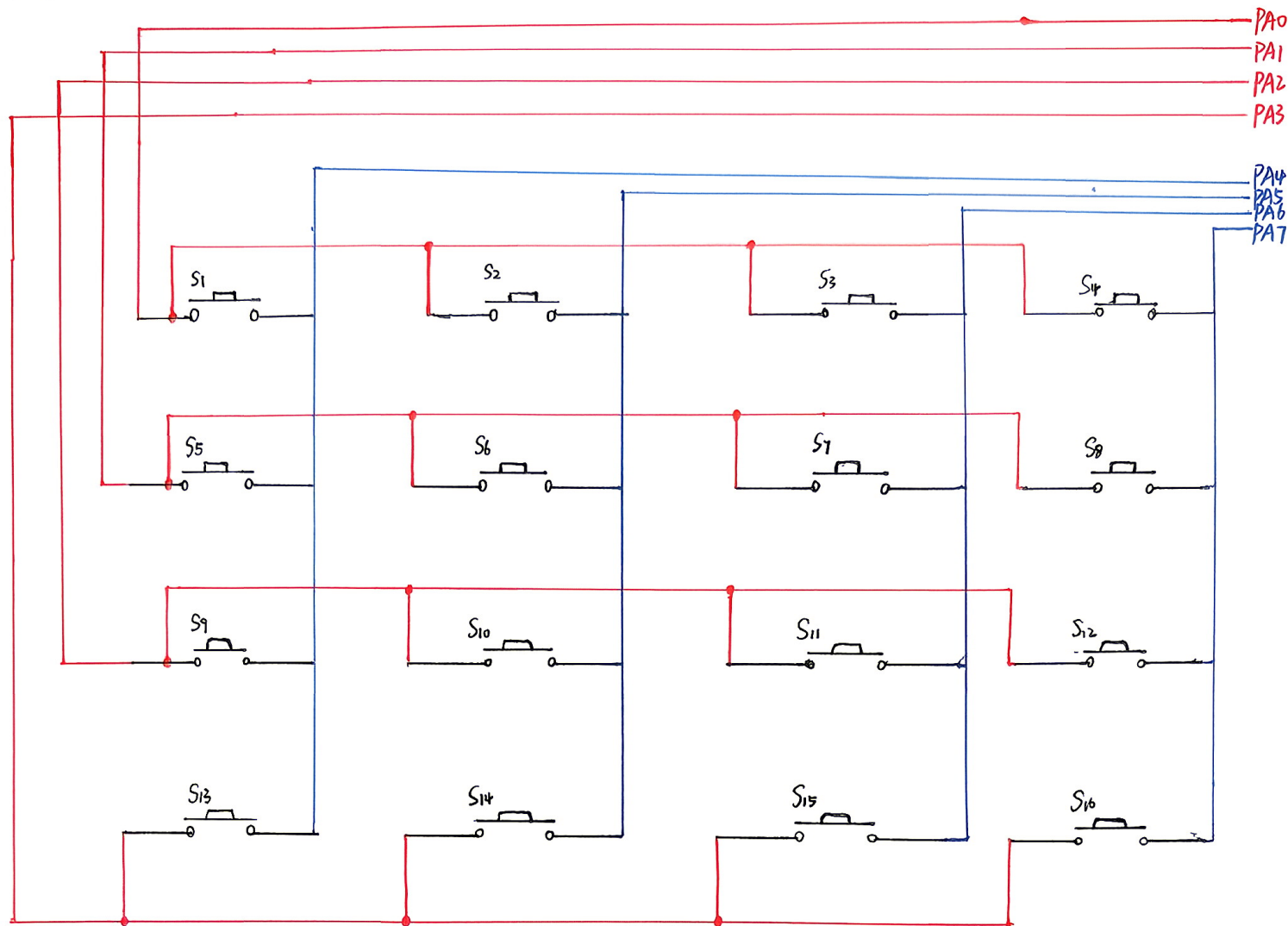
⑤ 两位位选关闭

⑥ 将十位数字写入ODR

⑦ 开启十位的位选

⑧ 延时5ms

矩阵键盘



特定的按键按下就会接通特定的行线和列线

⇒

确定接通的行、列线的两种方法 < 反转法
行列扫描法

⇒ 确定接通的行列线 ⇒ 找出操作了哪个按键

例如, S1 按下就接通了 PA0, PA4.

反转法:

- ① 四行输出高电平, 检查列线输入, 确定列线
- ② 四列输出高电平, 检查行线输入, 确定行线

行列扫描法:

逐行(列)输出高电平, 检查列(行)线是否有高电平输入, 若有, 同时确定行和列.

输出控制方式

① 直接写 ODR 寄存器

② 通过 BRR, BSRR 寄存器写 ODR

③ 通过库函数写 ODR

// 初始化 PA0, PA1 为推挽式输出

GPIOA → CRL &= 0xFFFFF00;
 32位寄存器, 要8个16进制位对其进行操作

GPIOA → CRL |= 0x0000000B;
 2MHz 50MHz 00: 推挽输出

// PA0 输出低电平, PA1 输出高电平

注意其它位状态不变!

GPIOA → ODR &= 0xFFFC;

将 PA0, PA1 对应的 ODR 寄存器的位清 0

GPIOA → ODR |= 0x0002;

将 PA0 对应的寄存器位写 1, 将 PA1 对应的寄存器位写 1.

GPIOA → BRR |= 0x00000001;

设置端口位清除寄存器的低 16 位, 向 BRR 写入 1, 清除对应 ODR 位为 0

GPIOA → BSRR |= 0x00000002;

设置端口位设置/清除寄存器的低 16 位, 对 BS 写入 1, 执行 Set 功能

将 PA1 引脚对应的寄存器位置 1

ODR 1 位

GPIO_ResetBits(GPIOA, GPIO_Pin_0);

GPIO_SetBits(GPIOA, GPIO_Pin_1);

寄存器地址映射

GPIOA → ODR = 0x00000000;

值 0x00000000 如何赋给 GPIOA 的 ODR 寄存器地址?

GPIOA → ODR 如何与 GPIOA 的 ODR 寄存器地址映射起来?

GPIO 寄存器的偏移量

每一组 GPIO 端口有 1 个寄存器, 它们的地址排序方式由本组 GPIO 基地址算起, 每个寄存器依次加 4 个偏移量排列.

000h → 004h → 008h → 00ch → 010h → 014h → 018h

层层嵌套的关系:

外设基地址

⋮

APB2 基地址

⋮

GPIOA 基地址

⋮

#define PERIPH_BASE ((uint32_t)0x40000000)

当给出某寄存器地址时, 往往以“基地址 + 偏移量”形式出现, 我们可通过逐步查找基地址值的方式计算其具体地址

可转换为数值

#define APB2PERIPH_BASE (PERIPH_BASE + 0x10000)

#define GPIOA_BASE (APB2PERIPH_BASE + 0x0800)

#define GPIOA ((GPIO_TypeDef*)GPIOA_BASE)

typedef struct {
 __IO uint32_t

CRL;
CRH;
IDR;
ODR;
BSRR;
BRR;
LCKR;

将 GPIOA 定义为指向 GPIOA 基地址的结构体指针的类型

将 GPIOA 的 7 个寄存器定义为一个结构体

} GPIO_TypeDef;

位绑定

寄存器 32 个位 $\rightarrow$ 32 个对应地址

原理：把每个比特位膨胀为一个 32 位的字，访问这些字时就达到了访问比特的目的

比如，BSRR 寄存器有 32 个位，可映射到 32 个地址上，我们访问（读-改写）这 32 个地址就达到访问 32 个比特的目的。

映射关系：

① 位带区（支持位带操作的地址区）

② 位带别名（对别名地址的访问最终作用到位带区的访问）

SRAM 区最低 1MB 范围 $0x20000000 \sim 0x200FFFFF$ 与 片上外设区最低 1MB 范围

字节

$0x40000000 \sim 0x400FFFFF$ 支持位操作。

1 $\rightarrow$ 32 的关系
位带别的区的空间

优点：

① 简化操作

由位操作转为字操作，降低复杂度，简化操作过程

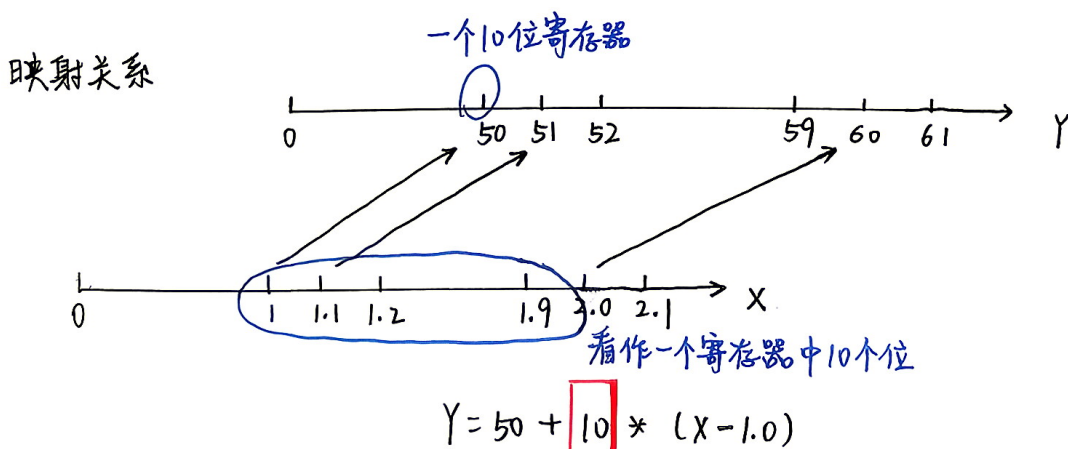
② 提高指令执行速度

针对直接操作 ODR 寄存器的情况，GPIO 输出操作过程中，先要读 ODR 数据，对要修改的位清 0，再写入新的值，最后写回寄存器；而位绑定可直接对字写入，实现对寄存器中位的操作。

③ 执行过程原子性

对寄存器中某一位的操作历经多个过程，若中途发生中断，恰巧也对该寄存器操作，则极有可能产生数据错误，位绑定可一次性完成对数据的操作

已知位带区寄存器地址，计算别名区地址的方式：



膨胀系数

GPIOA 基地址 + 偏移量

假设一个寄存器处于外设位带区，地址为 A，位数为 n，计算其变量的别名地址

这样，对 PA0 操作等同于对别名区操作。

别名区起始地址

当前寄存器地址

32 位 = 4 字节

为了对该地址直接赋值

$$\text{AliasAddr} = 0x42000000 + 32 * (A - 0x40000000) + 4 * n$$

该寄存器 data[0] 映射后的地址

ODR 偏移量

寄存器中具体某一位偏移情况

计算 GPIOA 的 ODRb0 的别名地址（别名区寄存器仅 60 位有效）
不能对地址直接赋值，要转换成（volatile unsigned long *）
指针类型，形成指向该地址的指针
define PA0_out Y(0x42000000 + 32 * (GPIOA - BASE + 0x0C - 0x40000000) + 4 * 0)

```
#define PA0out *((volatile unsigned long*)(0x42000000+(GPIOA-BASE+0X0C-0X40000000)*32+0*4))
```

```
#define PA1out *((volatile unsigned long*)(0x42000000+(GPIOA-BASE+0X0C-0X40000000)*32+1*4))
```

```
GPIOA → CRL &= 0xFFFFFFFF00;
```

```
GPIOA → CRL | = 0x00000023;
```

```
PA0out = 0;
```

```
PA1out = 1;
```

别名区的寄存器只有60位有效!